

容器 Container

第 5 讲

目标

把镜像这张"图纸", 盖成一间能住的"房子"

- 本质: 镜像的**运行实例** · vs 虚拟机谁更轻、为何轻
- 生命周期: `run/ps/stop/start/restart/rm` 全流程
- 操作: `exec/logs/top/stats/inspect`
- 两个实践: nginx 全流程 · 隔离与数据持久化

一、什么是容器

镜像的"运行实例"，本质是一个被隔离的进程

镜像是图纸，容器是照图纸盖出的房子

维度	镜像 Image	容器 Container
时机	构建时 构造	运行时 构造
本质	只读模板	模板的可运行实例
可写性	只读、不可改	可读可写可执行可挂卷
生命周期	持久	可启动、可停止、可销毁

盖坏了拆掉重建, 图纸(镜像)不受影响

容器不是"小操作系统"，是和宿主机共享内核的进程

对比项	虚拟机 VM	容器 Container
隔离级别	硬件级 虚拟化	内核级 隔离
是否自带内核	每个 VM 带完整内核	没有内核 , 共享宿主内核
体积	以 GB 计	以 MB 计
启动速度	分钟级	秒级

记住一句话: **容器就是进程**——被隔离的普通进程

"像台小机器", 靠 namespace 看清楚细节

namespace	隔离了什么
PID	进程号(容器内从 1 开始)
Mount	文件系统挂载点
Network	网络栈(独立网卡/IP/端口)
UTS	主机名
IPC	进程间通信
User	用户和 UID

容器的三个典型特征

- 📁 可写: 只读镜像之上叠一层可写容器层(写时复制)
- 🏠 隔离: 容器之间、与宿主机默认互不干扰
- 🔨 易销毁: 廉价一次性, 用坏删掉重建

二、生命周期管理

创建 → 启动 → 停止/启动 → 删除

docker run 创建并启动容器

```
docker run [参数] <镜像> [命令]
docker run hello-world # 体验一把
docker run -it --name ubuntu1 ubuntu:22.04 # 交互式进容器
docker run -d --name web1 -p 8080:80 nginx # 后台+端口映射
```



- `-i / -t` 交互式伪终端 · `-d` 后台 · `-p` 主机:容器
- `run` = **create** → **start** → **附加运行** 三层动作

ps 查看 · stop/start 启停 · rm 删除

```
docker ps                # 只看运行中的
docker ps -a             # 含已停止的

docker stop web1         # 优雅停止(先 SIGTERM)
docker start web1        # 启动已停止的容器
docker restart web1      # 重启

docker rm web1           # 删已停止; rm -f 强删运行中
```



stop 是"给时间善后"的优雅停止, kill 是直接 SIGKILL 强杀

生命周期全景图



三、容器常用操作

看日志 · 进容器 · 看进程 · 读元数据

logs 看日志 · exec 进容器内部

```
docker logs myweb          # 查看输出
docker logs -f myweb       # 持续跟踪(Ctrl+C 退出)
docker logs --tail 50 myweb # 只看最后 50 行

docker exec -it myweb /bin/bash # 进入运行中的容器
docker exec myweb ls /         # 跑单条命令后退出
```



exec 是"另起一个进程", 不重启容器; 后台(-d)容器靠 logs 观测

top 看进程 · stats 看资源 · inspect 读元数据

```
docker top myweb          # 容器内运行的进程
docker stats              # 实时 CPU/内存/网络/IO

docker inspect myweb      # 全部配置(JSON)
docker inspect -f '{{.State.Status}}' myweb # 模板取字段
```



端口映射 + 卷挂载：两个"桥"

```
docker run -d -p 8080:80 nginx # 宿主机8080 → 容器80
```



- 容器有独立网络栈, 宿主机**不能直接访问**其端口, 靠 -p 桥接

```
docker run -d --name db -v /宿主机:/容器 mysql # 挂载目录  
docker volume ls ; docker volume inspect <卷名>
```



容器可写层随容器删除而消失 → 想持久化必须用卷

四、完整实例

nginx 容器全流程

一条命令串起前面全部核心命令

一条龙： run → ps → logs → curl → exec → stop → rm

```
docker run -d --name myweb -p 8080:80 nginx # ① 后台运行
docker ps # ② 看状态
docker logs myweb # ③ 看日志
curl http://localhost:8080 # ④ 访问→欢迎页
docker exec -it myweb /bin/bash # ⑤ 进容器
    ls /usr/share/nginx/html ; cat /etc/os-release
    exit # ⑥ 退出不删
docker stop myweb ; docker rm myweb # ⑦ 停止删除
docker ps -a # ⑧ 确认已删
```



五、实战一

跑起你的第一个应用容器

拉取 → 前后台 → 进入 → 停止删除, 建立第一手感

前后台对比：一个 Ctrl+C 的差别

```
docker pull nginx:latest          # 引发拉取
docker run nginx:latest           # ① 前台：终端被占住
#                                ↑ Ctrl+C 会向容器发停止信号

docker run -d --name myweb -p 8080:80 nginx:latest # ② 后台
docker ps                         # STATUS 显示 Up
curl http://localhost:8080        # ③ 应得默认欢迎页
```



前台 Ctrl+C 停; 后台 curl 访问 + logs 看记录

进容器探一探：容器 = "小机器"的实感

```
docker exec -it myweb /bin/bash
ls /usr/share/nginx/html      # 容器网站根目录
hostname                      # 返回容器 ID!
ps aux                        # 只看到容器内进程
cat /etc/os-release           # 容器内系统(debian)
exit                          # 退出，容器继续跑
```



停止删除：容器没了, 镜像还在

```
docker stop myweb
docker ps      # 已不在(停止的不显示)
docker ps -a   # 以 Exited 出现在这里
docker rm myweb # 真正删除
docker images  # 镜像还在!
```



删的是"房子", 图纸(镜像)安然无恙

六、实战二

两容器隔离 + 数据持久化

同镜像是"各自为政", 数据是"删则全无"

同一个镜像, 两个互不干扰的容器

```
docker run -d --name site1 -p 8081:80 nginx
docker run -d --name site2 -p 8082:80 nginx
docker exec -it site1 /bin/bash
echo "Hello from site1" > /usr/share/nginx/html/index.html
exit

curl http://localhost:8081    # 显示 Hello from site1
curl http://localhost:8082    # 仍是默认首页!
```



痛点：容器数据不持久, 删就没了

```
docker rm -f site1 site2
docker run -d --name site1 -p 8081:80 nginx # 重建
curl http://localhost:8081 # 刚才改的内容没了!
```



关键认知：可写层随容器销毁。想保留数据, 必须用卷

数据卷: 容器没了, 数据跨容器存活

```
docker volume create webdata
docker run -d --name site1 -v webdata:/usr/share/nginx/html -p 8081:80 nginx
docker exec -it site1 /bin/bash
echo "Persistent!" > /usr/share/nginx/html/index.html ; exit
docker rm -f site1 ; docker volume ls      # 卷还在

docker run -d --name site2 -v webdata:/usr/share/nginx/html -p 8082:80 nginx
curl http://localhost:8082    # 显示 Persistent!—数据回来了
```



一张表带走全部要点

方面	要点
本质	镜像的 运行实例 , 本质是被隔离的进程; 无内核共享宿主内核
镜像 vs 容器	构建时模板 vs 运行时实例; 一图启 N 房
生命周期	create→start→stop→rm; 配 restart、ps -a 查状态
核心命令	run(-it/-d/-p/-name) · ps · stop/start · rm; 辅以 exec/logs/top/stats/inspect
关键概念	-d+logs; exec -it; -p 端口桥; 数据不持久用卷

容器会"跑", 可它连不清自己人

后续内容: 卷(Volume)、网络(Network)与 Compose 编排——让多个容器协作成一套应用

谢谢大家

Q & A